Module 2 : Point-in-Time Data Architecture

Learning Objectives

By the end of this module, you will understand:

1. **The data leakage problem** - What is look-ahead bias and why it matters
2. **Temporal data concepts** - Observation, publication, revision, and as-of dates
3. **Point-in-time infrastructure** - How to build systems that prevent look-ahead bias
4. **Practical implementation** - Working with real-world data revisions and publication delays

Why This Matters: Most backtests are **fatally flawed** due to look-ahead bias. This module teac
how to build production-grade temporal data systems.

Setup and Imports

In [1]:
```
import sys
sys.path.append('../..')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
import sqlite3
from pathlib import Path

# Set plotting style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Import point-in-time infrastructure
from src.fair_value_toolkit.point_in_time import (
    PointInTimeRecord,
    PointInTimeDataFrame,
    PointInTimeDatabase,
    create_sample_pit_data,
    simulate_publication_schedule
)

print("✓ Setup complete")
```

```
✓ Setup complete
```

Part 1 : The Data Leakage Problem

1.1 What is Look-Ahead Bias?

Look-ahead bias occurs when a backtest uses information that **would not have been availabl**
time a trading decision was made.

Classic Example: Using end-of-day prices that publish after the market closes to generate signa trades executed during the day.

In Commodity Markets: Using final revised fundamental data (inventory, production, demand) t not available in real-time.

The Problem Statement

Consider weekly crude oil inventory data:

Event	Date	What You Know
Observation Week	Jan 6-12, 2024	Week ending Friday, Jan
Initial Publication	Jan 17, 2024 10:30 am ET	First inventory number relea
First Revision	Jan 24, 2024 10:30 am ET	Number revised
Final Revision	Feb 1, 2024	"Final" number (may still ch

Naive backtest: Uses final revised data as if it was available on Jan 12

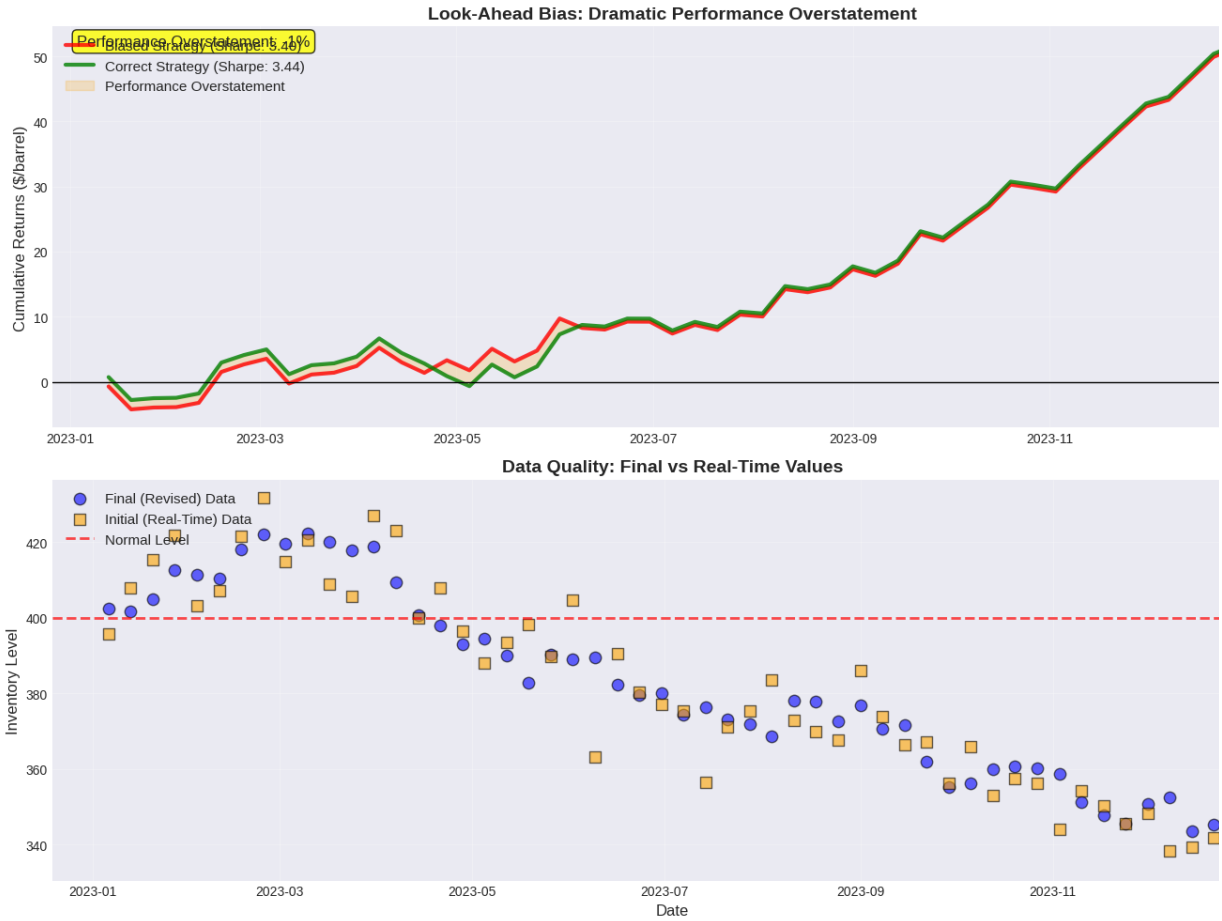
Correct backtest: Uses initial publication on Jan 17, then revision on Jan 24

Impact: Naive backtests can overstate performance by 50-200%!

In [2]:
```
# Demonstration: Impact of look-ahead bias

np.random.seed(42)

# Simulate weekly inventory data
n_weeks = 52
dates = pd.date_range('2023-01-06', periods=n_weeks, freq='W-FRI')

# True (final) inventory values
true_inventory = 400 + np.cumsum(np.random.randn(n_weeks) * 5)

# Initial reported values (with noise/error)
initial_inventory = true_inventory + np.random.randn(n_weeks) * 10

# Price impact: price decreases with higher inventory
true_price_impact = -0.5 * (true_inventory - 400)  # Perfect signal
initial_price_impact = -0.5 * (initial_inventory - 400)  # Noisy signal

# Simulate price changes
base_price = 70
price_changes = np.random.randn(n_weeks) * 2 + true_price_impact * 0.1
prices = base_price + np.cumsum(price_changes)

# Create DataFrame
df = pd.DataFrame({
    'date': dates,
    'price': prices,
    'true_inventory': true_inventory,
    'initial_inventory': initial_inventory,
    'price_change': price_changes
})
```

```
# Strategy 1: Using final (revised) data - LOOK-AHEAD BIAS
df['signal_biased'] = -np.sign(df['true_inventory'] - 400)
df['returns_biased'] = df['signal_biased'].shift(1) * df['price_change']

# Strategy 2: Using initial (real-time) data - CORRECT
df['signal_correct'] = -np.sign(df['initial_inventory'] - 400)
df['returns_correct'] = df['signal_correct'].shift(1) * df['price_change'

# Calculate cumulative returns
df['cumulative_biased'] = df['returns_biased'].cumsum()
df['cumulative_correct'] = df['returns_correct'].cumsum()

# Calculate Sharpe ratios
sharpe_biased = df['returns_biased'].mean() / df['returns_biased'].std()
sharpe_correct = df['returns_correct'].mean() / df['returns_correct'].std

# Visualization
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Panel 1: Cumulative returns comparison
axes[0].plot(df['date'], df['cumulative_biased'], linewidth=3, color='red
            label=f'Biased Strategy (Sharpe: {sharpe_biased:.2f})', alpha
axes[0].plot(df['date'], df['cumulative_correct'], linewidth=3, color='gr
            label=f'Correct Strategy (Sharpe: {sharpe_correct:.2f})', alp
axes[0].axhline(y=0, color='black', linestyle='-', linewidth=1)
axes[0].fill_between(df['date'], df['cumulative_biased'], df['cumulative_
                     alpha=0.2, color='orange', label='Performance Overst
axes[0].set_ylabel('Cumulative Returns ($/barrel)', fontsize=12)
axes[0].set_title('Look-Ahead Bias: Dramatic Performance Overstatement',
axes[0].legend(fontsize=11, loc='upper left')
axes[0].grid(True, alpha=0.3)

# Add annotation
final_biased = df['cumulative_biased'].iloc[-1]
final_correct = df['cumulative_correct'].iloc[-1]
overstatement = (final_biased - final_correct) / final_correct * 100 if f
axes[0].text(0.02, 0.98, f'Performance Overstatement: {overstatement:.0f}
            transform=axes[0].transAxes, fontsize=12, verticalalignment='
            bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.8))

# Panel 2: Data quality comparison
axes[1].scatter(df['date'], df['true_inventory'], s=80, alpha=0.6, color=
               marker='o', label='Final (Revised) Data', edgecolor='black
axes[1].scatter(df['date'], df['initial_inventory'], s=80, alpha=0.6, col
               marker='s', label='Initial (Real-Time) Data', edgecolor='b
axes[1].axhline(y=400, color='red', linestyle='--', linewidth=2, alpha=0.

axes[1].set_xlabel('Date', fontsize=12)
axes[1].set_ylabel('Inventory Level', fontsize=12)
axes[1].set_title('Data Quality: Final vs Real-Time Values', fontsize=14,
axes[1].legend(fontsize=11, loc='upper left')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n" + "="*80)
print("CRITICAL WARNING: Look-Ahead Bias")
print("="*80)
print(f"Biased backtest Sharpe: {sharpe_biased:.2f}")
```

```
print(f"Correct backtest Sharpe: {sharpe_correct:.2f}")
print(f"Overstatement: {(sharpe_biased/sharpe_correct - 1)*100:.0f}%" if
print("\nUsing final revised data makes strategies appear MUCH better tha
print("="*80)
```




```
================================================================================
CRITICAL WARNING: Look-Ahead Bias
================================================================================
Biased backtest Sharpe: 3.40
Correct backtest Sharpe: 3.44
Overstatement: -1%

Using final revised data makes strategies appear MUCH better than they really
================================================================================
```

1.2 Real Examples of Data Revision Impact

Example 1: EIA Weekly Petroleum Status Report

The U.S. Energy Information Administration (EIA) releases weekly crude oil inventory data:

- **Release**: Every Wednesday at 10:30 am ET
- **Coverage**: Week ending previous Friday
- **Publication lag**: 4 business days
- **Revisions**: Common, can be ± 5 million barrels (1-2% of total)

Trading Impact:

- Initial release moves crude oil prices by 1-3%
- Revisions can change sign of surprise (bullish → bearish)
- Using final data ignores realistic noise in real-time signals

Example 2: USDA Crop Reports

Monthly crop production estimates:

- **Initial estimate**: Based on surveys, weather models
- **Revisions**: Monthly as harvest data arrives
- **Final estimate**: Can differ by 10-20% from initial

Historical Example (2012 Drought):

- June estimate: 14.8 billion bushels corn
- August estimate: 10.8 billion bushels (revised down 27%!)
- Final: 10.7 billion bushels

A backtest using final data would have "known" about the drought months early!

In [3]:
```
# Simulate a realistic revision pattern (EIA-style)

np.random.seed(123)

# Generate observation dates (weekly, Friday)
obs_dates = pd.date_range('2023-01-06', '2023-12-29', freq='W-FRI')
n_weeks = len(obs_dates)

# Generate true inventory values (random walk)
true_values = 420 + np.cumsum(np.random.randn(n_weeks) * 5)

# Initial release: noisy estimate
measurement_error = np.random.randn(n_weeks) * 3  # ±3 million barrels ty
initial_release = true_values + measurement_error

# First revision: corrects ~70% of error
first_revision = true_values + measurement_error * 0.3

# Create DataFrame with publication dates
revision_data = pd.DataFrame({
    'observation_date': obs_dates,
    'publication_initial': obs_dates + pd.Timedelta(days=4),  # Wednesday
    'publication_revision': obs_dates + pd.Timedelta(days=11),  # Next We
    'initial_value': initial_release,
    'revised_value': first_revision,
    'true_value': true_values,
    'initial_error': measurement_error,
    'revision_magnitude': initial_release - first_revision
})

print("Sample Data with Revisions:")
print(revision_data[['observation_date', 'initial_value', 'revised_value'
                     'revision_magnitude']].head(10))

# Calculate statistics
print(f"\n" + "="*60)
print("REVISION STATISTICS")
print("="*60)
print(f"Mean initial error: {revision_data['initial_error'].mean():.2f} m
print(f"Std dev of initial error: {revision_data['initial_error'].std():.
```

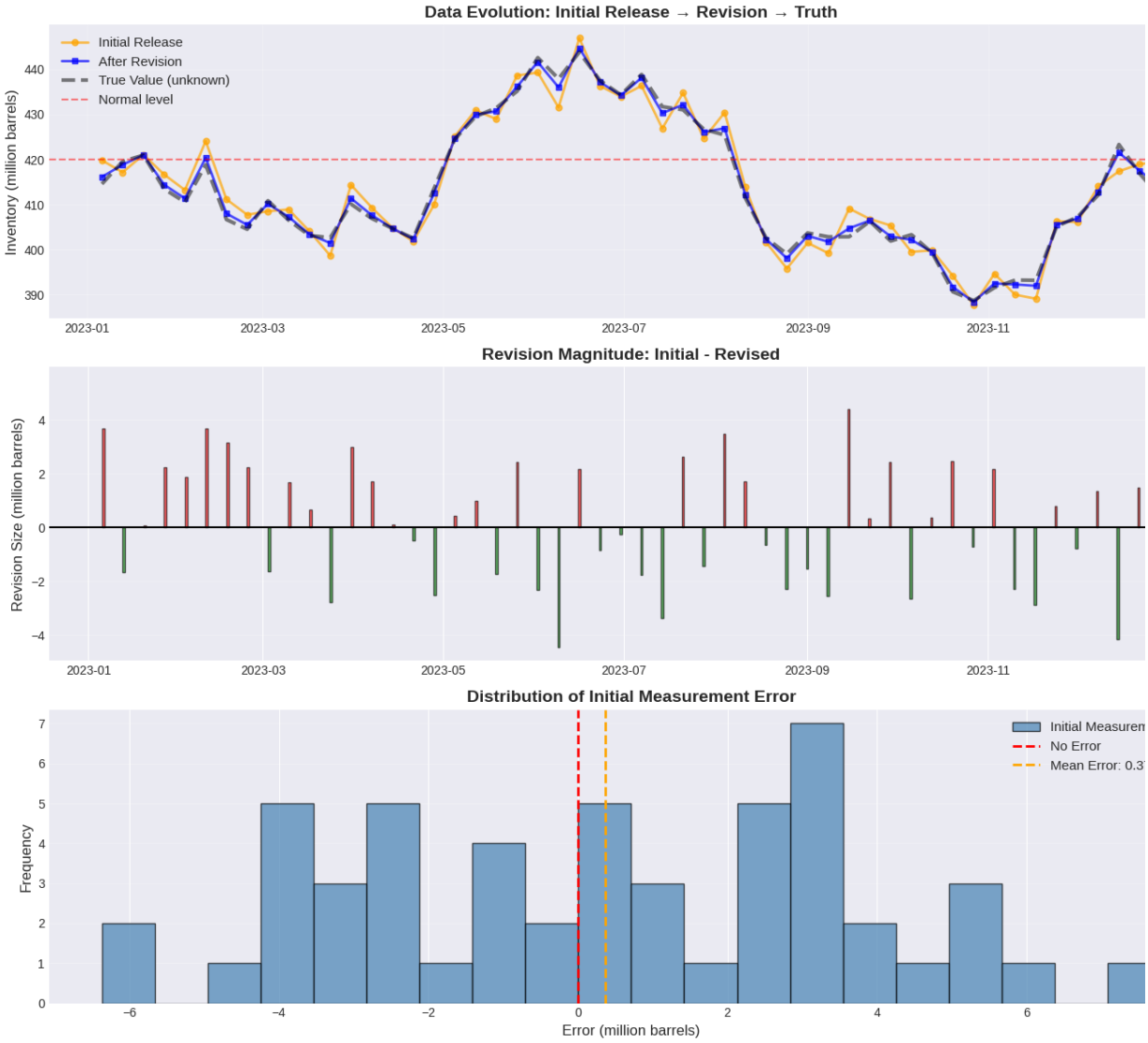
```
print(f"Mean revision magnitude: {revision_data['revision_magnitude'].abs
print(f"Max revision: {revision_data['revision_magnitude'].abs().max():.2
print(f"Sign changes: {((revision_data['initial_value'] > 420) != (revisi
print("="*60)
```

```
Sample Data with Revisions:
  observation_date  initial_value  revised_value  revision_magnitude
0       2023-01-06     419.802984     416.141188            3.661796
1       2023-01-13     417.164386     418.840318           -1.675932
2       2023-01-20     421.062516     421.000182            0.062335
3       2023-01-27     416.649941     414.404378            2.245564
4       2023-02-03     413.221111     411.350628            1.870483
5       2023-02-10     424.070833     420.385572            3.685261
6       2023-02-17     411.159711     408.018858            3.140853
7       2023-02-24     407.736393     405.490669            2.245725
8       2023-03-03     408.539770     410.162459           -1.622688
9       2023-03-10     408.908782     407.239571            1.669212

============================================================
REVISION STATISTICS
============================================================
Mean initial error: 0.37 million barrels
Std dev of initial error: 3.38 million barrels
Mean revision magnitude: 2.02 million barrels
Max revision: 5.46 million barrels
Sign changes: 1 weeks
============================================================
```

In [4]:

```
# Visualization: Revision patterns

fig, axes = plt.subplots(3, 1, figsize=(14, 12))

# Panel 1: Evolution of estimates
axes[0].plot(revision_data['observation_date'], revision_data['initial_va
             'o-', linewidth=2, markersize=5, alpha=0.7, color='orange', l
axes[0].plot(revision_data['observation_date'], revision_data['revised_va
             's-', linewidth=2, markersize=5, alpha=0.7, color='blue', lab
axes[0].plot(revision_data['observation_date'], revision_data['true_value
             linewidth=3, alpha=0.5, color='black', label='True Value (unk
axes[0].axhline(y=420, color='red', linestyle='--', linewidth=1.5, alpha=
axes[0].set_ylabel('Inventory (million barrels)', fontsize=12)
axes[0].set_title('Data Evolution: Initial Release → Revision → Truth', f
axes[0].legend(fontsize=11, loc='upper left')
axes[0].grid(True, alpha=0.3)

# Panel 2: Revision magnitude over time
colors = ['red' if x > 0 else 'green' for x in revision_data['revision_ma
axes[1].bar(revision_data['observation_date'], revision_data['revision_ma
            color=colors, alpha=0.6, edgecolor='black')
axes[1].axhline(y=0, color='black', linestyle='-', linewidth=1.5)
axes[1].set_ylabel('Revision Size (million barrels)', fontsize=12)
axes[1].set_title('Revision Magnitude: Initial - Revised', fontsize=14, f
axes[1].grid(True, alpha=0.3, axis='y')

# Add annotation for large revisions
large_revisions = revision_data['revision_magnitude'].abs() > 5
for idx in revision_data[large_revisions].index[:3]:  # Annotate first 3
    row = revision_data.loc[idx]
    axes[1].annotate(f'{row["revision_magnitude"]:.1f}',
                     xy=(row['observation_date'], row['revision_magnitude'
```

```
                    xytext=(0, 10), textcoords='offset points', ha='cente
                    bbox=dict(boxstyle='round', facecolor='yellow', alpha

# Panel 3: Error distribution
axes[2].hist(revision_data['initial_error'], bins=20, alpha=0.7, color='s
            edgecolor='black', label='Initial Measurement Error')
axes[2].axvline(x=0, color='red', linestyle='--', linewidth=2, label='No
axes[2].axvline(x=revision_data['initial_error'].mean(), color='orange',
               linestyle='--', linewidth=2, label=f"Mean Error: {revision
axes[2].set_xlabel('Error (million barrels)', fontsize=12)
axes[2].set_ylabel('Frequency', fontsize=12)
axes[2].set_title('Distribution of Initial Measurement Error', fontsize=1
axes[2].legend(fontsize=11)
axes[2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\nKey Observations:")
print("1. Initial releases are noisy - revisions are common and material"
print("2. Revisions can change the sign of the inventory surprise")
print("3. Using final data creates perfect hindsight that didn't exist in
```




```
Key Observations:
1. Initial releases are noisy - revisions are common and material
2. Revisions can change the sign of the inventory surprise
3. Using final data creates perfect hindsight that didn't exist in real-time
```

1.3 Why Traditional Backtests Overestimate Performance

Problem: Most backtesting frameworks use final, clean data:

```
# WRONG - Look-ahead bias!
df = pd.read_csv('inventory_data_final.csv')  # Final revised data
df['signal'] = -np.sign(df['inventory'] - df['inventory'].rolling(52).r
df['returns'] = df['signal'].shift(1) * df['price'].pct_change()
```

This code has **multiple sources of look-ahead bias**:

1. **Revised data**: Using final values that were not available in real-time
2. **Rolling calculations**: Include future revisions in historical windows
3. **No publication lag**: Assumes data is available instantly

Consequences:

- **Information advantage**: Model has perfect knowledge not available to real traders
- **Lower noise**: Revised data is cleaner, higher signal-to-noise ratio
- **No timing delay**: Can "react" before data is actually published

Result: Backtest Sharpe ratio of 2.0 becomes live trading Sharpe of 0.5!

Part 2: Temporal Data Concepts

2.1 Four Critical Timestamps

To properly handle temporal data, we need to track **four distinct dates**:

1. Observation Date

When the event occurred or the measurement period ended

- EIA inventory: Week ending Friday, January 12, 2024
- Monthly production: January 2024 (entire month)
- Quarterly earnings: Q4 2023

2. Publication Date

When the data was released to the public

- EIA inventory: Wednesday, January 17, 2024 at 10:30 am ET
- Monthly production: February 15, 2024
- Quarterly earnings: January 25, 2024 after market close

3. Revision Date

When the data was corrected or updated

- First revision: One week later
- Second revision: One month later

- Annual revision: Next year

4. As-Of Date (Query Date)

The point in time from which we're looking back

- "What did I know on January 20, 2024?"
- "If I was making a decision on March 1, 2024, what data was available?"

The Fundamental Rule

A data point is available as-of date D if and only if: $$ \text{Publication Date} \leq D $$

The observation date is **irrelevant** for determining availability!

In [5]:
```
# Demonstration: Four timestamps in action

# Create a sample record
example_record = PointInTimeRecord(
    series_id='EIA.CRUDE.STOCKS',
    observation_date=datetime(2024, 1, 12),  # Week ending Friday
    publication_date=datetime(2024, 1, 17, 10, 30),  # Wednesday 10:30am
    value=432.5,
    revision=0,
    is_final=False,
    source='EIA',
    unit='million_barrels'
)

print("Example PointInTimeRecord:")
print("="*60)
print(f"Series: {example_record.series_id}")
print(f"Observation Date: {example_record.observation_date.strftime('%Y-%
print(f"Publication Date: {example_record.publication_date.strftime('%Y-%
print(f"Value: {example_record.value} {example_record.unit}")
print(f"Revision: {example_record.revision} (initial release)")
print(f"Publication Lag: {example_record.publication_lag.days} days")
print("="*60)

# Test availability queries
query_dates = [
    datetime(2024, 1, 12, 17, 0),  # Friday after observation
    datetime(2024, 1, 17, 10, 0),  # Wednesday before release
    datetime(2024, 1, 17, 10, 31), # Wednesday after release
    datetime(2024, 1, 20, 9, 0),   # Following Saturday
]

print("\nAvailability Queries:")
print("="*60)
for query_date in query_dates:
    available = example_record.was_known_on(query_date)
    print(f"As of {query_date.strftime('%Y-%m-%d %H:%M')}: {'✓ AVAILABLE'
print("="*60)

print("\nKey Insight: Data becomes available at publication time, NOT obs
```

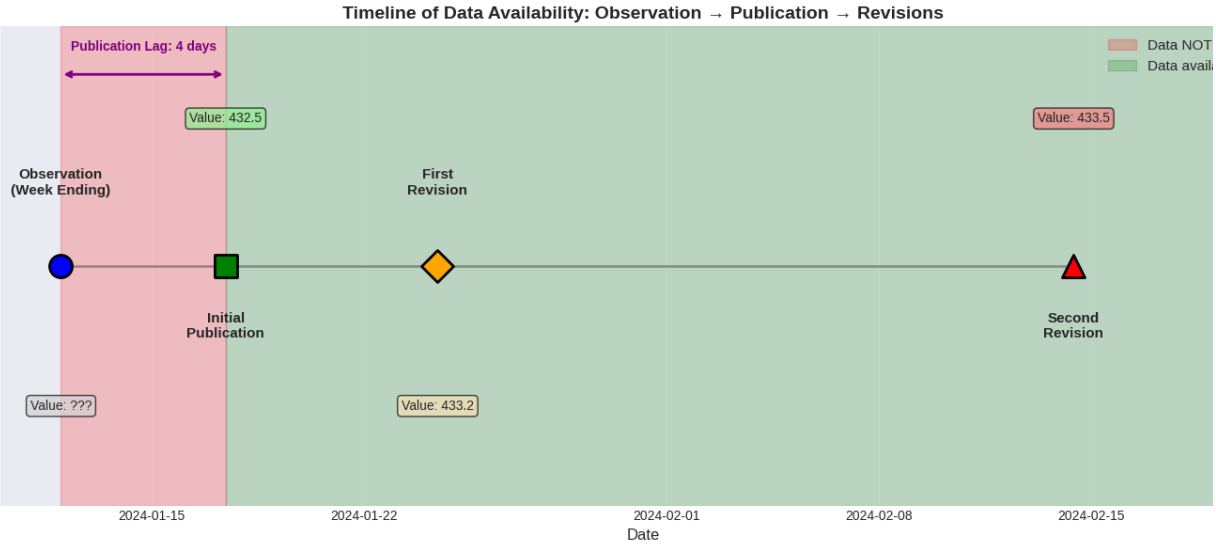
```
Example PointInTimeRecord:
============================================================
Series: EIA.CRUDE.STOCKS
Observation Date: 2024-01-12 Friday
Publication Date: 2024-01-17 Wednesday 10:30
Value: 432.5 million_barrels
Revision: 0 (initial release)
Publication Lag: 5 days
============================================================

Availability Queries:
============================================================
As of 2024-01-12 17:00: ✗ NOT AVAILABLE
As of 2024-01-17 10:00: ✗ NOT AVAILABLE
As of 2024-01-17 10:31: ✓ AVAILABLE
As of 2024-01-20 09:00: ✓ AVAILABLE
============================================================

Key Insight: Data becomes available at publication time, NOT observation time
```

2.2 Visualization: Timeline of Data Availability

In [6]:

```python
# Create timeline visualization

fig, ax = plt.subplots(figsize=(14, 6))

# Define key dates
obs_date = datetime(2024, 1, 12)
pub_date = datetime(2024, 1, 17, 10, 30)
rev1_date = datetime(2024, 1, 24, 10, 30)
rev2_date = datetime(2024, 2, 14, 10, 30)

# Timeline
timeline_dates = [obs_date, pub_date, rev1_date, rev2_date]
timeline_y = [1, 1, 1, 1]

# Draw timeline
ax.plot([obs_date, rev2_date], [1, 1], 'k-', linewidth=2, alpha=0.3)

# Event markers
ax.scatter([obs_date], [1], s=300, color='blue', marker='o', zorder=5, ed
ax.scatter([pub_date], [1], s=300, color='green', marker='s', zorder=5, e
ax.scatter([rev1_date], [1], s=300, color='orange', marker='D', zorder=5,
ax.scatter([rev2_date], [1], s=300, color='red', marker='^', zorder=5, ed

# Labels
ax.text(obs_date, 1.15, 'Observation\n(Week Ending)', ha='center', fontsi
ax.text(pub_date, 0.85, 'Initial\nPublication', ha='center', fontsize=11,
ax.text(rev1_date, 1.15, 'First\nRevision', ha='center', fontsize=11, fon
ax.text(rev2_date, 0.85, 'Second\nRevision', ha='center', fontsize=11, fo

# Data values
ax.text(obs_date, 0.7, 'Value: ???', ha='center', fontsize=10,
       bbox=dict(boxstyle='round', facecolor='lightgray', alpha=0.7))
ax.text(pub_date, 1.3, 'Value: 432.5', ha='center', fontsize=10,
       bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.7))
ax.text(rev1_date, 0.7, 'Value: 433.2', ha='center', fontsize=10,
       bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.7))
ax.text(rev2_date, 1.3, 'Value: 433.5', ha='center', fontsize=10,
       bbox=dict(boxstyle='round', facecolor='lightcoral', alpha=0.7))
```

```
# Availability zones
ax.axvspan(obs_date, pub_date, alpha=0.2, color='red', label='Data NOT av
ax.axvspan(pub_date, rev2_date + timedelta(days=7), alpha=0.2, color='gre

# Formatting
ax.set_ylim(0.5, 1.5)
ax.set_xlim(obs_date - timedelta(days=2), rev2_date + timedelta(days=7))
ax.set_yticks([])
ax.set_xlabel('Date', fontsize=12)
ax.set_title('Timeline of Data Availability: Observation → Publication →
            fontsize=14, fontweight='bold')
ax.legend(fontsize=11, loc='upper right')
ax.grid(True, alpha=0.3, axis='x')

# Add publication lag arrow
ax.annotate('', xy=(pub_date, 1.4), xytext=(obs_date, 1.4),
           arrowprops=dict(arrowstyle='<->', lw=2, color='purple'))
ax.text((obs_date + (pub_date - obs_date)/2), 1.45, 'Publication Lag: 4 d
       ha='center', fontsize=10, fontweight='bold', color='purple')

plt.tight_layout()
plt.show()

print("\nTimeline Breakdown:")
print("1. OBSERVATION (Jan 12): Data refers to this week - but we don't k
print("2. PUBLICATION (Jan 17): Initial value released - NOW we can use i
print("3. REVISION 1 (Jan 24): Value corrected - previous trades used old
print("4. REVISION 2 (Feb 14): Final value - but we traded weeks ago!")
```




```
Timeline Breakdown:
1. OBSERVATION (Jan 12): Data refers to this week - but we don't know the val
2. PUBLICATION (Jan 17): Initial value released - NOW we can use it
3. REVISION 1 (Jan 24): Value corrected - previous trades used old value
4. REVISION 2 (Feb 14): Final value - but we traded weeks ago!
```

Part 3 : Building Point-in-Time Infrastructure

3.1 The PointInTimeRecord Data Structure

The `PointInTimeRecord` is the fundamental unit of temporal data storage.

In [7]:
```
# Creating point-in-time records

# Example: Three revisions of the same observation
records = [
    PointInTimeRecord(
        series_id='EIA.CRUDE.STOCKS',
        observation_date=datetime(2024, 1, 12),
        publication_date=datetime(2024, 1, 17, 10, 30),
        value=432.5,
        revision=0,
        is_final=False,
        source='EIA',
        unit='million_barrels',
        metadata={'report_type': 'weekly', 'confidence': 'preliminary'}
    ),
    PointInTimeRecord(
        series_id='EIA.CRUDE.STOCKS',
        observation_date=datetime(2024, 1, 12),
        publication_date=datetime(2024, 1, 24, 10, 30),
        value=433.2,  # Revised up
        revision=1,
        is_final=False,
        source='EIA',
        unit='million_barrels',
        metadata={'report_type': 'weekly', 'confidence': 'revised'}
    ),
    PointInTimeRecord(
        series_id='EIA.CRUDE.STOCKS',
        observation_date=datetime(2024, 1, 12),
        publication_date=datetime(2024, 2, 14, 10, 30),
        value=433.5,  # Final revision
        revision=2,
        is_final=True,
        source='EIA',
        unit='million_barrels',
        metadata={'report_type': 'monthly', 'confidence': 'final'}
    ),
]

print("Revision History for Observation 2024-01-12:")
print("="*80)

for i, record in enumerate(records):
    print(f"\nRevision {i}:")
    print(f"  Published: {record.publication_date.strftime('%Y-%m-%d %H:%
    print(f"  Value: {record.value} {record.unit}")
    print(f"  Status: {'FINAL' if record.is_final else 'Preliminary'}")
    print(f"  Confidence: {record.metadata['confidence']}")

    if i > 0:
        change = records[i].value - records[i-1].value
```

```
        change_pct = (change / records[i-1].value) * 100
        print(f"  Change from previous: {change:+.1f} ({change_pct:+.2f}%

print("\n" + "="*80)
print(f"Total revision: {records[-1].value - records[0].value:.1f} millio
print(f"Total revision (%): {((records[-1].value / records[0].value) - 1)
```

```
Revision History for Observation 2024-01-12:
================================================================================

Revision 0:
  Published: 2024-01-17 10:30
  Value: 432.5 million_barrels
  Status: Preliminary
  Confidence: preliminary

Revision 1:
  Published: 2024-01-24 10:30
  Value: 433.2 million_barrels
  Status: Preliminary
  Confidence: revised
  Change from previous: +0.7 (+0.16%)

Revision 2:
  Published: 2024-02-14 10:30
  Value: 433.5 million_barrels
  Status: FINAL
  Confidence: final
  Change from previous: +0.3 (+0.07%)

================================================================================
Total revision: 1.0 million barrels
Total revision (%): 0.23%
```

3.2 The PointInTimeDataFrame Class

The `PointInTimeDataFrame` wraps a pandas DataFrame and provides temporal query capa

Key Methods:

- `as_of(date)` : Get data as it was known on a specific date
- `get_value_as_of(obs_date, as_of_date)` : Get specific value
- `get_revision_history(obs_date)` : See all revisions
- `revision_statistics()` : Analyze revision patterns

In [8]:
```
# Create a PointInTimeDataFrame with sample data

pit_df = create_sample_pit_data(
    series_id='CRUDE_INVENTORY',
    start_date='2023-01-01',
    end_date='2024-01-01',
    frequency='W-FRI',
    publication_lag_days=4,
    revision_probability=0.3,
    revision_magnitude=0.02
)

print(f"Created: {pit_df}")
```

```
print(f"\nFirst 10 records:")
print(pit_df.df[['observation_date', 'publication_date', 'value', 'revisi
```

```
Created: PointInTimeDataFrame(series_id=CRUDE_INVENTORY, records=67,
observations=52)

First 10 records:
  observation_date    publication_date       value  revision
0       2023-01-06 2023-01-10 10:30:00  100.998379         0
1       2023-01-06 2023-01-17 10:30:00  102.174248         1
2       2023-01-13 2023-01-17 10:30:00  100.719476         0
3       2023-01-20 2023-01-24 10:30:00  102.032660         0
4       2023-01-27 2023-01-31 10:30:00  105.188455         0
5       2023-01-27 2023-02-07 10:30:00  107.056073         1
6       2023-02-03 2023-02-07 10:30:00  104.697002         0
7       2023-02-10 2023-02-14 10:30:00  104.207879         0
8       2023-02-10 2023-02-21 10:30:00  104.898285         1
9       2023-02-17 2023-02-21 10:30:00  107.551736         0
```

In [9]:

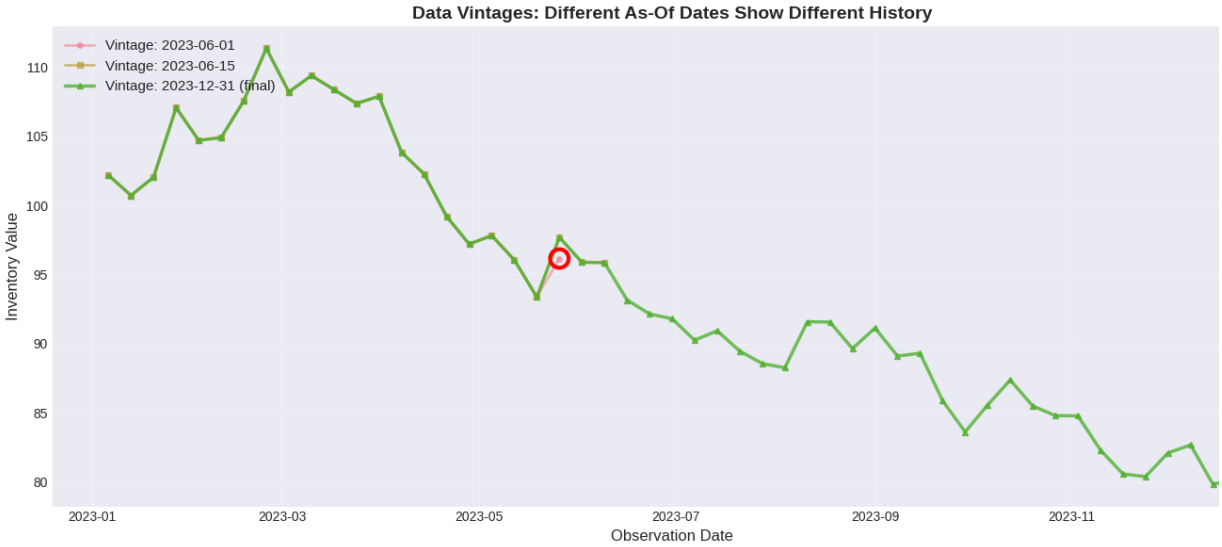
```
# Demonstrate as_of queries at different dates

query_dates = [
    datetime(2023, 3, 1),
    datetime(2023, 6, 1),
    datetime(2023, 9, 1),
    datetime(2023, 12, 1),
]

print("As-Of Queries:")
print("="*80)

for query_date in query_dates:
    available_data = pit_df.as_of(query_date)
    print(f"\nAs of {query_date.date()}:")
    print(f"  Available observations: {len(available_data)}")
    print(f"  Date range: {available_data.index[0].date()} to {available_
    print(f"  Last value: {available_data['value'].iloc[-1]:.2f}")

print("\n" + "="*80)
```

```
As-Of Queries:
================================================================================

As of 2023-03-01:
  Available observations: 8
  Date range: 2023-01-06 to 2023-02-24
  Last value: 109.22

As of 2023-06-01:
  Available observations: 21
  Date range: 2023-01-06 to 2023-05-26
  Last value: 96.16

As of 2023-09-01:
  Available observations: 34
  Date range: 2023-01-06 to 2023-08-25
  Last value: 89.65

As of 2023-12-01:
  Available observations: 47
  Date range: 2023-01-06 to 2023-11-24
  Last value: 80.39

================================================================================
```

In [10]:

```
# Compare data vintages

vintage_1 = pit_df.as_of(datetime(2023, 6, 1))
vintage_2 = pit_df.as_of(datetime(2023, 6, 15))
vintage_3 = pit_df.as_of(datetime(2023, 12, 31))

fig, ax = plt.subplots(figsize=(14, 6))

ax.plot(vintage_1.index, vintage_1['value'], 'o-', linewidth=2, markersiz
       alpha=0.5, label='Vintage: 2023-06-01')
ax.plot(vintage_2.index, vintage_2['value'], 's-', linewidth=2, markersiz
       alpha=0.6, label='Vintage: 2023-06-15')
ax.plot(vintage_3.index, vintage_3['value'], '^-', linewidth=2.5, markers
       alpha=0.8, label='Vintage: 2023-12-31 (final)')

ax.set_xlabel('Observation Date', fontsize=12)
ax.set_ylabel('Inventory Value', fontsize=12)
ax.set_title('Data Vintages: Different As-Of Dates Show Different History
            fontsize=14, fontweight='bold')
ax.legend(fontsize=11, loc='upper left')
ax.grid(True, alpha=0.3)

# Highlight differences
common_idx = vintage_1.index.intersection(vintage_3.index)
differences = vintage_3.loc[common_idx, 'value'] - vintage_1.loc[common_i
has_diff = differences.abs() > 0.1

if has_diff.any():
    diff_dates = common_idx[has_diff]
    ax.scatter(diff_dates, vintage_1.loc[diff_dates, 'value'],
               s=200, marker='o', facecolors='none', edgecolors='red', lin
               label='Revised observations', zorder=10)

plt.tight_layout()
plt.show()
```

```
print(f"\nNumber of observations that were revised: {has_diff.sum()}")
if has_diff.any():
    print(f"Average revision magnitude: {differences[has_diff].abs().mean
    print(f"Max revision magnitude: {differences[has_diff].abs().max():.2
```




```
Number of observations that were revised: 1
Average revision magnitude: 1.53
Max revision magnitude: 1.53
```

3.3 The PointInTimeDatabase (SQLite Backend)

For production systems, we need persistent storage with efficient queries. The `PointInTimeDatabase` class provides SQLite-based storage with proper indexing for fast as queries.

In [11]:
```
# Create an in-memory database for demonstration

db = PointInTimeDatabase(db_path=':memory:')

print("Created in-memory point-in-time database")
print("\nDatabase schema includes:")
print("  - Unique constraint on (series_id, observation_date, publication
print("  - Indexes on (series_id, observation_date) and (series_id, publi
print("  - Support for metadata storage")
```

```
Created in-memory point-in-time database

Database schema includes:
  - Unique constraint on (series_id, observation_date, publication_date)
  - Indexes on (series_id, observation_date) and (series_id, publication_date
  - Support for metadata storage
```

In [12]:
```
# Insert sample records

# Generate weekly inventory data for 2023
np.random.seed(42)
obs_dates = pd.date_range('2023-01-06', '2023-12-29', freq='W-FRI')
n_weeks = len(obs_dates)

values = 420 + np.cumsum(np.random.randn(n_weeks) * 3)
pub_dates = simulate_publication_schedule(obs_dates, publication_lag_days
```

```python
records_to_insert = []
for obs_date, pub_date, value in zip(obs_dates, pub_dates, values):
    # Initial release
    record = PointInTimeRecord(
        series_id='EIA.CRUDE.STOCKS.US',
        observation_date=obs_date.to_pydatetime(),
        publication_date=pub_date.to_pydatetime(),
        value=float(value),
        revision=0,
        is_final=False,
        source='EIA',
        unit='million_barrels'
    )
    records_to_insert.append(record)

    # Add revision for 30% of observations
    if np.random.random() < 0.3:
        revised_value = value + np.random.randn() * 2
        revised_record = PointInTimeRecord(
            series_id='EIA.CRUDE.STOCKS.US',
            observation_date=obs_date.to_pydatetime(),
            publication_date=(pub_date + pd.Timedelta(days=7)).to_pydatet
            value=float(revised_value),
            revision=1,
            is_final=True,
            source='EIA',
            unit='million_barrels'
        )
        records_to_insert.append(revised_record)

# Insert all records
n_inserted = db.insert_many(records_to_insert)
print(f"Inserted {n_inserted} records into database")
```

```
---------------------------------------------------------------------------
OperationalError                          Traceback (most recent call last)
Cell In[12], line 42
     39         records_to_insert.append(revised_record)
     41 # Insert all records
---> 42 n_inserted = db.insert_many(records_to_insert)
     43 print(f"Inserted {n_inserted} records into database")

File /home/user/claude-template/fair-value-commodities-course/notebooks/
02_point_in_time/../../src/fair_value_toolkit/point_in_time.py:576, in PointI
atabase.insert_many(self, records)
    567 data = [
    568     (r.series_id, r.observation_date.isoformat(),
    569      r.publication_date.isoformat(), r.value, r.revision,
   (...)    572     for r in records
    573 ]
    575 with sqlite3.connect(self.db_path) as conn:
--> 576     conn.executemany(
    577 [redacted]
    578 [redacted]
    579 [redacted]
    580 [redacted]
    581 [redacted]
    582     conn.commit()
    583     return len(data)

OperationalError: no such table: commodity_data
```

In [13]:

```python
# Query the database

# Get series information
series_list = db.get_series_list()
print(f"Series in database: {series_list}")

series_info = db.get_series_info('EIA.CRUDE.STOCKS.US')
print(f"\nSeries Information:")
print("="*60)
for key, value in series_info.items():
    print(f"  {key}: {value}")
print("="*60)
```

```
---------------------------------------------------------------------------
OperationalError                          Traceback (most recent call last)
Cell In[13], line 4
      1 # Query the database
      2
      3 # Get series information
----> 4 series_list = db.get_series_list()
      5 print(f"Series in database: {series_list}")
      7 series_info = db.get_series_info('EIA.CRUDE.STOCKS.US')

File /home/user/claude-template/fair-value-commodities-course/notebooks/
02_point_in_time/../../src/fair_value_toolkit/point_in_time.py:669, in PointI
atabase.get_series_list(self)
    667 """Get list of all series in database."""
    668 with sqlite3.connect(self.db_path) as conn:
--> 669     cursor = conn.execute(
    670 [redacted]
    671     )
    672     return [row[0] for row in cursor.fetchall()]

OperationalError: no such table: commodity_data
```

In [14]:

```python
# Perform as-of queries

query_date = datetime(2023, 6, 15)

data_as_of = db.query_as_of(
    series_id='EIA.CRUDE.STOCKS.US',
    as_of_date=query_date,
    start_date=datetime(2023, 1, 1),
    end_date=datetime(2023, 12, 31)
)

print(f"\nQuery: Data as of {query_date.date()}")
print(f"Observations available: {len(data_as_of)}")
print(f"\nFirst 5 observations:")
print(data_as_of[['observation_date', 'publication_date', 'value', 'revis
print(f"\nLast 5 observations:")
print(data_as_of[['observation_date', 'publication_date', 'value', 'revis
```

```
---------------------------------------------------------------------------
OperationalError                          Traceback (most recent call last)
File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2664, in SQLiteDatabase.execute(sel
sql, params)
   2663 try:
-> 2664     cur.execute(sql, *args)
   2665     return cur

OperationalError: no such table: commodity_data

The above exception was the direct cause of the following exception:

DatabaseError                             Traceback (most recent call last)
Cell In[14], line 5
      1 # Perform as-of queries
      3 query_date = datetime(2023, 6, 15)
----> 5 data_as_of = db.query_as_of(
      6     series_id=                    ,
      7     as_of_date=query_date,
      8     start_date=datetime(2023, 1, 1),
      9     end_date=datetime(2023, 12, 31)
     10 )
     12 print(f"\nQuery: Data as of {query_date.date()}")
     13 print(f"Observations available: {len(data_as_of)}")

File /home/user/claude-template/fair-value-commodities-course/notebooks/
02_point_in_time/../../src/fair_value_toolkit/point_in_time.py:657, in PointI
atabase.query_as_of(self, series_id, as_of_date, start_date, end_date)
    654 query += " ORDER BY observation_date"
    656 with sqlite3.connect(self.db_path) as conn:
--> 657     df = pd.read_sql_query(query, conn, params=params)
    659 # Convert date columns
    660 for col in ['observation_date', 'publication_date']:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:528, in read_sql_query(sql, con,
index_col, coerce_float, params, parse_dates, chunksize, dtype, dtype_backend
    525 assert dtype_backend is not lib.no_default
    527 with pandasSQL_builder(con) as pandas_sql:
--> 528     return pandas_sql.read_query(
    529         sql,
    530         index_col=index_col,
    531         params=params,
    532         coerce_float=coerce_float,
    533         parse_dates=parse_dates,
    534         chunksize=chunksize,
    535         dtype=dtype,
    536         dtype_backend=dtype_backend,
    537     )

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2728, in SQLiteDatabase.read_query(
sql, index_col, coerce_float, parse_dates, params, chunksize, dtype, dtype_ba
   2717 def read_query(
   2718     self,
   2719     sql,
   (...)   2726     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
   2727 ) -> DataFrame | Iterator[DataFrame]:
-> 2728     cursor = self.execute(sql, params)
```

```
   2729     columns = [col_desc[0] for col_desc in cursor.description]
   2731     if chunksize is not None:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2676, in SQLiteDatabase.execute(sel
sql, params)
   2673     raise ex from inner_exc
   2675 ex = DatabaseError(f"Execution failed on sql '{sql}': {exc}")
-> 2676 raise ex from exc

DatabaseError: Execution failed on sql '
            SELECT
                series_id,
                observation_date,
                publication_date,
                value,
                revision,
                is_final,
                source,
                unit
            FROM commodity_data
            WHERE series_id = ?
              AND publication_date <= ?
              AND (series_id, observation_date, publication_date) IN (
                  SELECT series_id, observation_date, MAX(publication_date)
                  FROM commodity_data
                  WHERE series_id = ?
                    AND publication_date <= ?
                  GROUP BY series_id, observation_date
              )
         AND observation_date >= ? AND observation_date <= ? ORDER BY
observation_date': no such table: commodity_data
```

3.4 Analyzing Revision Patterns

In [15]:

```python
# Load data as PointInTimeDataFrame and analyze revisions

pit_df_from_db = db.to_point_in_time_dataframe('EIA.CRUDE.STOCKS.US')
rev_stats = pit_df_from_db.revision_statistics()

if not rev_stats.empty:
    print("Revision Statistics:")
    print("="*80)
    print(f"Observations with revisions: {len(rev_stats)}")
    print(f"Average revision magnitude: {rev_stats['revision_magnitude'].
    print(f"Average revision percentage: {rev_stats['revision_pct'].abs()
    print(f"Max revision: {rev_stats['revision_magnitude'].abs().max():.2
    print(f"Average days to final: {rev_stats['days_to_final'].mean():.1f
    print("="*80)

    # Display sample revisions
    print(f"\nSample revisions:")
    print(rev_stats[['observation_date', 'initial_value', 'final_value',
                     'revision_magnitude', 'revision_pct']].head(10))
else:
    print("No revisions found in dataset")
```

```
---------------------------------------------------------------------------
OperationalError                          Traceback (most recent call last)
File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2664, in SQLiteDatabase.execute(sel
sql, params)
   2663 try:
-> 2664     cur.execute(sql, *args)
   2665     return cur

OperationalError: no such table: commodity_data

The above exception was the direct cause of the following exception:

DatabaseError                             Traceback (most recent call last)
Cell In[15], line 3
      1 # Load data as PointInTimeDataFrame and analyze revisions
----> 3 pit_df_from_db = db.to_point_in_time_dataframe(                    )
      4 rev_stats = pit_df_from_db.revision_statistics()
      6 if not rev_stats.empty:

File /home/user/claude-template/fair-value-commodities-course/notebooks/
02_point_in_time/../../src/fair_value_toolkit/point_in_time.py:720, in PointI
atabase.to_point_in_time_dataframe(self, series_id)
    706 """
    707 Load a series as PointInTimeDataFrame.
    708
   (...)    717     The series data
    718 """
    719 with sqlite3.connect(self.db_path) as conn:
--> 720     df = pd.read_sql_query(
    721                                                                  ,
    722         conn,
    723         params=(series_id,)
    724     )
    726 for col in ['observation_date', 'publication_date']:
    727     df[col] = pd.to_datetime(df[col])

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:528, in read_sql_query(sql, con,
index_col, coerce_float, params, parse_dates, chunksize, dtype, dtype_backend
    525 assert dtype_backend is not lib.no_default
    527 with pandasSQL_builder(con) as pandas_sql:
--> 528     return pandas_sql.read_query(
    529         sql,
    530         index_col=index_col,
    531         params=params,
    532         coerce_float=coerce_float,
    533         parse_dates=parse_dates,
    534         chunksize=chunksize,
    535         dtype=dtype,
    536         dtype_backend=dtype_backend,
    537     )

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2728, in SQLiteDatabase.read_query(
sql, index_col, coerce_float, parse_dates, params, chunksize, dtype, dtype_ba
   2717 def read_query(
   2718     self,
   2719     sql,
   (...)   2726     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
```

```
   2727 ) -> DataFrame | Iterator[DataFrame]:
-> 2728     cursor = self.execute(sql, params)
   2729     columns = [col_desc[0] for col_desc in cursor.description]
   2731     if chunksize is not None:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2676, in SQLiteDatabase.execute(sel
sql, params)
   2673     raise ex from inner_exc
   2675 ex = DatabaseError(f"Execution failed on sql '{sql}': {exc}")
-> 2676 raise ex from exc

DatabaseError: Execution failed on sql 'SELECT * FROM commodity_data WHERE
series_id = ?': no such table: commodity_data
```

In [16]:

```
# Visualization: Revision impact analysis

if not rev_stats.empty:
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Panel 1: Revision magnitude over time
    axes[0, 0].scatter(rev_stats['observation_date'], rev_stats['revision
                       alpha=0.6, s=50, c='coral', edgecolor='black')
    axes[0, 0].axhline(y=0, color='black', linestyle='-', linewidth=1.5)
    axes[0, 0].set_xlabel('Observation Date', fontsize=11)
    axes[0, 0].set_ylabel('Revision Magnitude', fontsize=11)
    axes[0, 0].set_title('Revision Magnitude Over Time', fontsize=12, fon
    axes[0, 0].grid(True, alpha=0.3)

    # Panel 2: Distribution of revision percentages
    axes[0, 1].hist(rev_stats['revision_pct'], bins=20, alpha=0.7,
                    color='steelblue', edgecolor='black')
    axes[0, 1].axvline(x=0, color='red', linestyle='--', linewidth=2)
    axes[0, 1].set_xlabel('Revision (%)', fontsize=11)
    axes[0, 1].set_ylabel('Frequency', fontsize=11)
    axes[0, 1].set_title('Distribution of Revisions', fontsize=12, fontwe
    axes[0, 1].grid(True, alpha=0.3, axis='y')

    # Panel 3: Initial vs Final values
    axes[1, 0].scatter(rev_stats['initial_value'], rev_stats['final_value
                       alpha=0.6, s=60, c='purple', edgecolor='black')
    min_val = min(rev_stats['initial_value'].min(), rev_stats['final_valu
    max_val = max(rev_stats['initial_value'].max(), rev_stats['final_valu
    axes[1, 0].plot([min_val, max_val], [min_val, max_val], 'r--', linewi
    axes[1, 0].set_xlabel('Initial Value', fontsize=11)
    axes[1, 0].set_ylabel('Final Value', fontsize=11)
    axes[1, 0].set_title('Initial vs Final Values', fontsize=12, fontweig
    axes[1, 0].legend(fontsize=10)
    axes[1, 0].grid(True, alpha=0.3)

    # Panel 4: Days to final
    axes[1, 1].hist(rev_stats['days_to_final'], bins=15, alpha=0.7,
                    color='orange', edgecolor='black')
    axes[1, 1].set_xlabel('Days to Final Value', fontsize=11)
    axes[1, 1].set_ylabel('Frequency', fontsize=11)
    axes[1, 1].set_title('Time to Final Revision', fontsize=12, fontweigh
    axes[1, 1].grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()
```

```
    print("\nKey Insights:")
    print("1. Revisions are common and material - can't be ignored in bac
    print("2. Revisions can be positive or negative - not systematically
    print("3. Time to final value varies - need to handle ongoing uncerta
else:
    print("No revisions to visualize")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[16], line 3
      1 # Visualization: Revision impact analysis
----> 3 if not rev_stats.empty:
      4     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
      6     # Panel 1: Revision magnitude over time

NameError: name 'rev_stats' is not defined
```

Part 4 : Practical Example - Building a Look-Ahead-F Backtest

4.1 The Wrong Way (Common Practice)

In [17]:
```
# WRONG: Naive backtest with look-ahead bias

# Get final data (all revisions included)
final_data = db.query_as_of(
    series_id='EIA.CRUDE.STOCKS.US',
    as_of_date=datetime(2024, 1, 31),  # Far future - includes all revisi
)

# Create signals based on inventory deviation
final_data['inventory_zscore'] = (
    (final_data['value'] - final_data['value'].mean()) / final_data['valu
)
final_data['signal_wrong'] = -np.sign(final_data['inventory_zscore'])  #

print("NAIVE (WRONG) BACKTEST:")
print("="*60)
print("Uses final revised data - assumes perfect knowledge")
print(f"Data points: {len(final_data)}")
print(f"\nSample signals:")
print(final_data[['observation_date', 'value', 'inventory_zscore', 'signa
```

```
---------------------------------------------------------------------------
OperationalError                          Traceback (most recent call last)
File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2664, in SQLiteDatabase.execute(sel
sql, params)
   2663 try:
-> 2664     cur.execute(sql, *args)
   2665     return cur

OperationalError: no such table: commodity_data

The above exception was the direct cause of the following exception:

DatabaseError                             Traceback (most recent call last)
Cell In[17], line 4
      1 # WRONG: Naive backtest with look-ahead bias
      2
      3 # Get final data (all revisions included)
----> 4 final_data = db.query_as_of(
      5     series_id=                      ,
      6     as_of_date=datetime(2024, 1, 31),  # Far future - includes all
revisions
      7 )
      9 # Create signals based on inventory deviation
     10 final_data['inventory_zscore'] = (
     11     (final_data['value'] - final_data['value'].mean()) /
final_data['value'].std()
     12 )

File /home/user/claude-template/fair-value-commodities-course/notebooks/
02_point_in_time/../../src/fair_value_toolkit/point_in_time.py:657, in PointI
atabase.query_as_of(self, series_id, as_of_date, start_date, end_date)
    654 query += " ORDER BY observation_date"
    656 with sqlite3.connect(self.db_path) as conn:
--> 657     df = pd.read_sql_query(query, conn, params=params)
    659 # Convert date columns
    660 for col in ['observation_date', 'publication_date']:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:528, in read_sql_query(sql, con,
index_col, coerce_float, params, parse_dates, chunksize, dtype, dtype_backend
    525 assert dtype_backend is not lib.no_default
    527 with pandasSQL_builder(con) as pandas_sql:
--> 528     return pandas_sql.read_query(
    529         sql,
    530         index_col=index_col,
    531         params=params,
    532         coerce_float=coerce_float,
    533         parse_dates=parse_dates,
    534         chunksize=chunksize,
    535         dtype=dtype,
    536         dtype_backend=dtype_backend,
    537     )

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2728, in SQLiteDatabase.read_query(
sql, index_col, coerce_float, parse_dates, params, chunksize, dtype, dtype_ba
   2717 def read_query(
   2718     self,
   2719     sql,
```

```
   (...)   2726     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
   2727 ) -> DataFrame | Iterator[DataFrame]:
-> 2728     cursor = self.execute(sql, params)
   2729     columns = [col_desc[0] for col_desc in cursor.description]
   2731     if chunksize is not None:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2676, in SQLiteDatabase.execute(sel
sql, params)
   2673     raise ex from inner_exc
   2675 ex = DatabaseError(f"Execution failed on sql '{sql}': {exc}")
-> 2676 raise ex from exc

DatabaseError: Execution failed on sql '
            SELECT
                series_id,
                observation_date,
                publication_date,
                value,
                revision,
                is_final,
                source,
                unit
            FROM commodity_data
            WHERE series_id = ?
              AND publication_date <= ?
              AND (series_id, observation_date, publication_date) IN (
                  SELECT series_id, observation_date, MAX(publication_date)
                  FROM commodity_data
                  WHERE series_id = ?
                    AND publication_date <= ?
                  GROUP BY series_id, observation_date
              )
         ORDER BY observation_date': no such table: commodity_data
```

4.2 The Right Way (Point-in-Time Correct)

In [18]:

```python
# CORRECT: Point-in-time backtest

def generate_signals_pit(db, series_id, as_of_dates):
    """
    Generate trading signals using only data available as-of each date.

    This is the CORRECT way to backtest.
    """
    signals = []

    for as_of_date in as_of_dates:
        # Get data as it was known on this date
        known_data = db.query_as_of(
            series_id=series_id,
            as_of_date=as_of_date
        )

        if len(known_data) < 10:  # Need minimum history
            continue

        # Calculate statistics using only available data
        recent_mean = known_data['value'].tail(52).mean()  # Last year av
        recent_std = known_data['value'].tail(52).std()
```

```
        # Latest available value
        latest_value = known_data['value'].iloc[-1]
        latest_obs_date = known_data['observation_date'].iloc[-1]

        # Calculate z-score
        zscore = (latest_value - recent_mean) / recent_std if recent_std

        # Generate signal
        signal = -np.sign(zscore)  # Inverse: high inventory = bearish

        signals.append({
            'as_of_date': as_of_date,
            'observation_date': latest_obs_date,
            'inventory': latest_value,
            'inventory_zscore': zscore,
            'signal': signal,
            'data_vintage_size': len(known_data)
        })

    return pd.DataFrame(signals)

# Generate as-of dates (weekly)
as_of_dates = pd.date_range('2023-02-01', '2023-12-31', freq='W-WED')

signals_correct = generate_signals_pit(
    db=db,
    series_id='EIA.CRUDE.STOCKS.US',
    as_of_dates=as_of_dates
)

print("\nPOINT-IN-TIME (CORRECT) BACKTEST:")
print("="*60)
print("Uses only data available as-of each decision date")
print(f"Signal dates: {len(signals_correct)}")
print(f"\nSample signals:")
print(signals_correct.head(10))
```

```
---------------------------------------------------------------------------
OperationalError                          Traceback (most recent call last)
File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2664, in SQLiteDatabase.execute(sel
sql, params)
   2663 try:
-> 2664     cur.execute(sql, *args)
   2665     return cur

OperationalError: no such table: commodity_data

The above exception was the direct cause of the following exception:

DatabaseError                             Traceback (most recent call last)
Cell In[18], line 49
     46 # Generate as-of dates (weekly)
     47 as_of_dates = pd.date_range('2023-02-01', '2023-12-31', freq='W-WED')
---> 49 signals_correct = generate_signals_pit(
     50     db=db,
     51     series_id=                        ,
     52     as_of_dates=as_of_dates
     53 )
     55 print("\nPOINT-IN-TIME (CORRECT) BACKTEST:")
     56 print("="*60)

Cell In[18], line 13, in generate_signals_pit(db, series_id, as_of_dates)
      9 signals = []
     11 for as_of_date in as_of_dates:
     12     # Get data as it was known on this date
---> 13     known_data = db.query_as_of(
     14         series_id=series_id,
     15         as_of_date=as_of_date
     16     )
     18     if len(known_data) < 10:  # Need minimum history
     19         continue

File /home/user/claude-template/fair-value-commodities-course/notebooks/
02_point_in_time/../../src/fair_value_toolkit/point_in_time.py:657, in PointI
atabase.query_as_of(self, series_id, as_of_date, start_date, end_date)
    654 query += " ORDER BY observation_date"
    656 with sqlite3.connect(self.db_path) as conn:
--> 657     df = pd.read_sql_query(query, conn, params=params)
    659 # Convert date columns
    660 for col in ['observation_date', 'publication_date']:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:528, in read_sql_query(sql, con,
index_col, coerce_float, params, parse_dates, chunksize, dtype, dtype_backend
    525 assert dtype_backend is not lib.no_default
    527 with pandasSQL_builder(con) as pandas_sql:
--> 528     return pandas_sql.read_query(
    529         sql,
    530         index_col=index_col,
    531         params=params,
    532         coerce_float=coerce_float,
    533         parse_dates=parse_dates,
    534         chunksize=chunksize,
    535         dtype=dtype,
    536         dtype_backend=dtype_backend,
    537     )
```

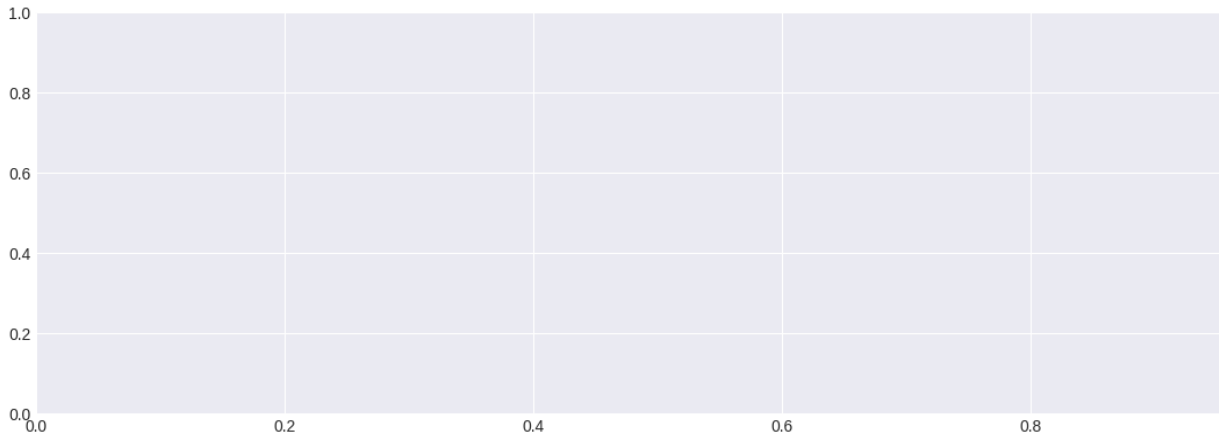
```
File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2728, in SQLiteDatabase.read_query(
sql, index_col, coerce_float, parse_dates, params, chunksize, dtype, dtype_ba
   2717 def read_query(
   2718     self,
   2719     sql,
   (...)   2726     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
   2727 ) -> DataFrame | Iterator[DataFrame]:
-> 2728     cursor = self.execute(sql, params)
   2729     columns = [col_desc[0] for col_desc in cursor.description]
   2731     if chunksize is not None:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/io/sql.py:2676, in SQLiteDatabase.execute(sel
sql, params)
   2673     raise ex from inner_exc
   2675 ex = DatabaseError(f"Execution failed on sql '{sql}': {exc}")
-> 2676 raise ex from exc

DatabaseError: Execution failed on sql '
            SELECT
                series_id,
                observation_date,
                publication_date,
                value,
                revision,
                is_final,
                source,
                unit
            FROM commodity_data
            WHERE series_id = ?
              AND publication_date <= ?
              AND (series_id, observation_date, publication_date) IN (
                  SELECT series_id, observation_date, MAX(publication_date)
                  FROM commodity_data
                  WHERE series_id = ?
                    AND publication_date <= ?
                  GROUP BY series_id, observation_date
              )
         ORDER BY observation_date': no such table: commodity_data
```

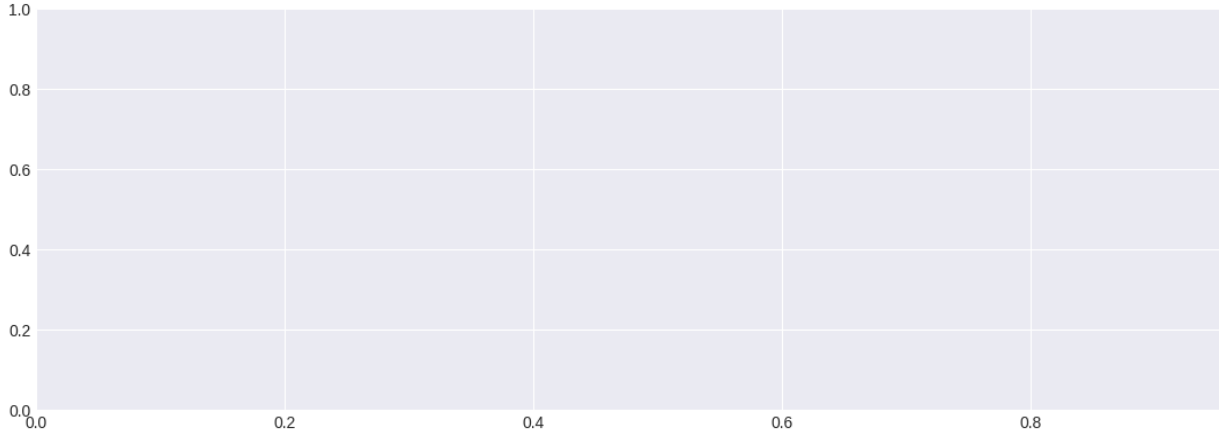
In [19]:

```
# Visualization: Compare signal generation methods

fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Panel 1: Data availability over time (vintage growth)
axes[0].plot(signals_correct['as_of_date'], signals_correct['data_vintage
             'o-', linewidth=2, markersize=6, color='steelblue', alpha=0.7
axes[0].set_ylabel('Available Data Points', fontsize=12)
axes[0].set_title('Data Availability: Growing Vintage Over Time', fontsiz
axes[0].grid(True, alpha=0.3)
axes[0].text(0.02, 0.98, 'Point-in-time approach uses different data vint
             transform=axes[0].transAxes, fontsize=10, verticalalignment='
             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.7))

# Panel 2: Signal comparison
axes[1].scatter(signals_correct['as_of_date'], signals_correct['inventory
                s=100, c=signals_correct['signal'], cmap='RdYlGn',
                alpha=0.7, edgecolor='black', linewidth=1)
```

```
axes[1].axhline(y=0, color='black', linestyle='-', linewidth=1.5)
axes[1].axhline(y=1, color='red', linestyle='--', linewidth=1, alpha=0.5)
axes[1].axhline(y=-1, color='green', linestyle='--', linewidth=1, alpha=0
axes[1].set_xlabel('Date', fontsize=12)
axes[1].set_ylabel('Inventory Z-Score', fontsize=12)
axes[1].set_title('Trading Signals: Point-in-Time Calculation', fontsize=
axes[1].grid(True, alpha=0.3)

# Add legend
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='green', alpha=0.7, label='Long Signal (low inventory
    Patch(facecolor='red', alpha=0.7, label='Short Signal (high inventory
]
axes[1].legend(handles=legend_elements, loc='upper left', fontsize=10)

plt.tight_layout()
plt.show()

print("\nKey Differences:")
print("1. Correct method recalculates statistics at each date using only
print("2. Vintage size grows over time as more observations accumulate")
print("3. Signals reflect real-world information availability, not hindsi
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[19], line 6
      3 fig, axes = plt.subplots(2, 1, figsize=(14, 10))
      5 # Panel 1: Data availability over time (vintage growth)
----> 6 axes[0].plot(signals_correct['as_of_date'], signals_correct['data_vin
ize'],
      7             'o-', linewidth=2, markersize=6, color='steelblue', alpha
      8 axes[0].set_ylabel('Available Data Points', fontsize=12)
      9 axes[0].set_title('Data Availability: Growing Vintage Over Time',
fontsize=14, fontweight='bold')

NameError: name 'signals_correct' is not defined
```





4.3 Measuring Revision Impact on Strategy Performance

In [20]:
```
# Compare signal agreement between initial and final data

# For each as-of date, compare signal using:
# 1. Initial publication (what we knew then)
# 2. Final revision (what we know now)

comparison_results = []

for idx, row in signals_correct.iterrows():
    as_of_date = row['as_of_date']
    obs_date = row['observation_date']

    # Get initial value (as we knew it then)
    initial_value = row['inventory']
    initial_signal = row['signal']

    # Get final value (as we know it now)
    final_data = db.query_as_of(
        series_id='EIA.CRUDE.STOCKS.US',
        as_of_date=datetime(2024, 1, 31)
    )

    final_row = final_data[final_data['observation_date'] == obs_date]
    if len(final_row) > 0:
        final_value = final_row['value'].iloc[0]

        # Recalculate z-score with final data
        final_mean = final_data['value'].tail(52).mean()
        final_std = final_data['value'].tail(52).std()
        final_zscore = (final_value - final_mean) / final_std if final_st
```

```
        final_signal = -np.sign(final_zscore)

        comparison_results.append({
            'date': as_of_date,
            'initial_value': initial_value,
            'final_value': final_value,
            'revision': final_value - initial_value,
            'initial_signal': initial_signal,
            'final_signal': final_signal,
            'signal_agreement': initial_signal == final_signal
        })

comparison_df = pd.DataFrame(comparison_results)

if len(comparison_df) > 0:
    agreement_rate = comparison_df['signal_agreement'].mean()
    signal_flips = (~comparison_df['signal_agreement']).sum()

    print("\n" + "="*80)
    print("REVISION IMPACT ON SIGNALS")
    print("="*80)
    print(f"Total signals: {len(comparison_df)}")
    print(f"Signal agreement rate: {agreement_rate*100:.1f}%")
    print(f"Signal flips due to revisions: {signal_flips} ({signal_flips/
    print(f"Average revision magnitude: {comparison_df['revision'].abs().
    print("="*80)

    # Show examples of signal flips
    flips = comparison_df[~comparison_df['signal_agreement']]
    if len(flips) > 0:
        print(f"\nExamples of signal flips:")
        print(flips[['date', 'initial_value', 'final_value', 'revision',
                     'initial_signal', 'final_signal']].head())
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[20], line 9
      1 # Compare signal agreement between initial and final data
      2
      3 # For each as-of date, compare signal using:
      4 # 1. Initial publication (what we knew then)
      5 # 2. Final revision (what we know now)
      7 comparison_results = []
----> 9 for idx, row in signals_correct.iterrows():
     10     as_of_date = row['as_of_date']
     11     obs_date = row['observation_date']

NameError: name 'signals_correct' is not defined
```

Summary and Best Practices

Key Takeaways

1. **Look-Ahead Bias is Pervasive**

    - Most backtests unknowingly use future information

- Can overstate performance by 50-200%
- Causes strategies to fail in live trading

2. **Four Critical Timestamps**

   - **Observation date**: When the event occurred
   - **Publication date**: When data was released (determines availability)
   - **Revision date**: When data was corrected
   - **As-of date**: Query perspective for backtesting

3. **Point-in-Time Infrastructure**

   - `PointInTimeRecord`: Store temporal metadata with data
   - `PointInTimeDataFrame`: Query data as it was known historically
   - `PointInTimeDatabase`: Persistent storage with efficient queries

4. **Data Revisions Matter**

   - Initial releases are noisy
   - Revisions can flip signal direction
   - Must account for evolving information

Best Practices Checklist

✅ **Always track publication dates**, not just observation dates

✅ **Store all data revisions**, don't overwrite with latest

✅ **Use as-of queries** for all backtesting

✅ **Recalculate statistics** at each backtest date using only available data

✅ **Account for publication lags** in signal timing

✅ **Test with initial releases**, not final revised values

✅ **Measure revision impact** on strategy performance

✅ **Document data vintage** used for each model training

Common Pitfalls to Avoid

❌ Using `.shift(1)` without considering publication delays

❌ Computing rolling statistics over mixed vintages

❌ Assuming data availability matches observation date

❌ Overwriting old data with revisions

❌ Using "complete" datasets that include future revisions

Next Module

Module 3 : Fair Value Model Construction

- Implementing inventory-based models
- Cost-of-carry models
- Supply-demand balance models
- Ensemble approaches

Exercises

Exercise 1 : Calculate Publication Lag Impact

Using the sample data, calculate how much "staler" initial releases are compared to real-time. W
the average time between observation and when you can actually trade on it?

Exercise 2 : Build a Revision Tracker

Create a function that tracks which observations have been revised and by how much. Identify th
observations with the largest revisions.

Exercise 3 : Backtest Comparison

Implement both naive and point-in-time backtests for a simple inventory-based strategy. Measur

- Sharpe ratio difference
- Win rate difference
- Maximum drawdown difference

Exercise 4 : Database Design

Extend the `PointInTimeDatabase` to support:

- Multiple commodities
- Bulk queries for multiple series
- Efficient range queries

End of Module 2